

claude-windows / 662b86ab-d2e3-4c02-8a0d-ec7a9071aba8

Metadata

```
- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\662b86ab-d2e3-4c02-8a0d-ec7a9071aba8\subagents\workflows\wf_498b24af-d16\agent-a91de3fc8e64fbfcd.jsonl`
- SHA-256: `dbdeffbf85e07bb1e21319f07d59b19c1949d4a5215ac767e1234ebbb785aa6e`
- Source modified: `2026-06-21T11:13:22+00:00`
- Imported at: `2026-07-05T16:48:27+00:00`
- project: `wf_498b24af-d16`
- session_id: `662b86ab-d2e3-4c02-8a0d-ec7a9071aba8`
```

Transcript

1. user (2026-06-21T11:10:34.620Z)

Review the COMPUTE_DECOUPLED_SERVING M8 change just committed on branch feat/serving-m8-cost-ledger (HEAD). Inspect with: `git diff origin/master...HEAD` and read: `backend/billing.py` (pure cost math `compute_cost_usd/apply_margin/to_inr`); `backend/infrastructure/database.py` (`_migrate_to_v6`: cost columns on jobs + pricebook table seeded placeholder-v0 at 0.0; `get_pricebook_rates`, `record_job_cost`, `get_job_cost`, `get_video_cost`); `backend/config/models.py` (`BillingConfig`, `default-OFF`); `backend/main.py` (`GET /api/jobs/{id}/cost + /api/videos/{id}/cost`); `backend/api/job_worker.py` (`_maybe_record_job_cost`, `gated`); `tests/test_billing.py` + the cost tests in `tests/test_api_endpoints.py`. CONTRACT: `DEFAULT-OFF` ? `billing.enabled=False` records nothing (cost columns stay `NULL` = today); the seeded placeholder-v0 pricebook makes every cost 0; the v6 migration is additive-`NULL` + idempotent. USD is the source of truth, `INR` a display conversion. Report ONLY real, grounded issues with `file:line`. Empty findings if a lens finds nothing real.

LENS = TEST RIGOR / API / HYGIENE. (1) Do the tests prove the ledger end-to-end or pass vacuously? Are BOTH the placeholder-0 path and a real-rate path tested? Is `default-OFF` (records nothing when disabled) AND the `enabled` path tested? (2) The API cost tests via `TestClient` ? do they assert the `INR` conversion + the 404 + null-before-recording? (3) `get_job_cost` SELECTs an explicit column list ? does it exactly match the v6 columns (typo risk)? (4) `_maybe_record_job_cost` reads `result['usage']` which the pipeline does not emit today, so cost is always 0 even when enabled until the worker emits usage ? is that honestly documented, not a hidden no-op? (5) `mypy/ruff` issues (the `billing` import, `Dict` typing, Optional handling)? (6) Any cheap missing test (unknown pricebook version yields 0; margin applied)? Find grounded test/API/hygiene gaps.

2. assistant (2026-06-21T11:10:37.381Z)

I'll review the M8 cost-ledger change with the TEST RIGOR / API / HYGIENE lens. Let me start by examining the diff and the relevant files.

3. user (2026-06-21T11:10:40.642Z)

582522b feat(serving): M8 ? per-job cost ledger + versioned pricebook + /cost API (default-OFF)

4. user (2026-06-21T11:10:40.854Z)

<code>backend/api/job_worker.py</code>	27 ++++++
<code>backend/billing.py</code>	60 ++++++
<code>backend/config/models.py</code>	16 ++++
<code>backend/infrastructure/database.py</code>	132 ++++++
<code>backend/main.py</code>	28 ++++++
<code>tests/test_api_endpoints.py</code>	46 ++++++
<code>tests/test_async_jobs.py</code>	8 +-
<code>tests/test_billing.py</code>	157 ++++++
<code>tests/test_database.py</code>	28 +++++

```
tests/test_job_worker_loop.py      | 8 +-
10 files changed, 490 insertions(+), 20 deletions(-)
```

5. assistant (2026-06-21T11:10:42.909Z)

Let me read the core files in the diff.

6. user (2026-06-21T11:10:44.043Z)

```
diff --git a/backend/api/job_worker.py b/backend/api/job_worker.py
index 1385c7a..d3ad660 100644
--- a/backend/api/job_worker.py
+++ b/backend/api/job_worker.py
@@ -88,6 +88,7 @@ def _run_claimed_job(job: Dict[str, Any]) -> None:
     if result.get("video_id"):
         db_instance.set_job_video_id(job_id, result["video_id"])
         db_instance.mark_job_done(job_id, result)
+
     _maybe_record_job_cost(job, result) # M8 cost ledger (default-OFF)
 except _main.JobWaiting as w:
     # Cooperative re-enqueue: persist progress + next_poll_at, release the worker.
     logger.info(f"Job {job_id} parked for {w.delay_sec:.0f}s (resume when due).")
@@ -97,6 +98,32 @@ def _run_claimed_job(job: Dict[str, Any]) -> None:
     db_instance.mark_job_failed(job_id, f"{type(e).__name__}: {e}")

+def _maybe_record_job_cost(job: Dict[str, Any], result: Dict[str, Any]) -> None:
+    """M8 cost ledger (COMPUTE_DECOUPLED_SERVING M8, D8). DEFAULT-OFF: a no-op unless
+    ``config.billing.enabled``. When on, compute the job's cost from the usage the pipeline
+    reported
+    (``result['usage']`` = {gpu_seconds, wall_seconds, egress_gb, gemini_usd, gpu_type}) x the
+    versioned pricebook, and persist it. With the placeholder pricebook every rate is 0 ? cost
+    0,
+    so turning billing on changes no user-facing number until the owner inserts real rates.
+
+    Best-effort ? never fails a completed job because cost bookkeeping hiccuped. ``job`` is the
+    claimed row (carries ``owner_id``); ``result`` is the pipeline output."""
+    import backend.main as _main
+
+    billing_cfg = getattr(_main.global_config, "billing", None)
+    if not billing_cfg or not getattr(billing_cfg, "enabled", False):
+        return
+    try:
+        usage = (result or {}).get("usage") or {}
+        _main.db_instance.record_job_cost(
+            job["id"], usage=usage, pricebook_version=billing_cfg.pricebook_version,
+            gpu_type=usage.get("gpu_type"), compute_backend=(result or
+            {}).get("compute_backend"),
+            owner_id=job.get("owner_id"), margin=billing_cfg.margin,
+        )
+        logger.info("Recorded cost for job %s (pricebook=%s).", job["id"],
+            billing_cfg.pricebook_version)
+    except Exception as e: # never wedge a completed job on bookkeeping
+        logger.warning("Cost recording for job %s failed (non-fatal): %s", job.get("id"), e)
+
+def _drain_due_jobs() -> int:
+    """One worker tick: claim and run every currently-runnable job (queued, or waiting whose
+    ``next_poll_at`` is due), earliest-due first, until the table has nothing runnable.
```

```
diff --git a/backend/billing.py b/backend/billing.py
new file mode 100644
index 0000000..c7f3445
--- /dev/null
+++ b/backend/billing.py
@@ -0,0 +1,60 @@
+"""Per-job cost ledger ? pure cost math (COMPUTE_DECOUPLED_SERVING M8, decision D8).
```

```

+
+`cost_usd(job) = ?_resource usage(job, resource) × rate(resource, pricebook_version)`
+(the formula from the archived 05-COST-ATTRIBUTION design). Cost is computed + stored in
**USD**
+(matches GCP billing = one source of truth) and **displayed in INR** at a configurable FX rate
+(the Bangalore market). The pricebook is a versioned DB table
(``backend/infrastructure/database.py``);
+this module is the pure arithmetic over a resolved ``{resource: unit_price_usd}`` rates dict,
so it
+is unit-tested with zero IO and the DB layer owns the version/gpu_type lookup.
+
+**Default-OFF:** with the seeded ``placeholder-v0`` pricebook (every rate ``0.0``) every cost
is
+``0.0`` ? nothing user-facing changes until the owner inserts a real, calibrated pricebook
version.
+"""
+
+
+from __future__ import annotations
+
+from typing import Dict, Mapping, Tuple
+
+
+## Metered resource keys ? shared by the usage dict, the pricebook rows, and the breakdown.
+GPU_SECONDS = "gpu_seconds"      # GPU-active seconds (the big one) ? rate is per-second, per
gpu_type
+WALL_SECONDS = "wall_seconds"    # total wall time (separate from GPU so idle/sync isn't billed
as GPU)
+EGRESS_GB = "egress_gb"         # reel/byte egress in GB
+GEMINI_USD = "gemini_usd"       # provider-reported cost, ALREADY in USD (a passthrough, not
rate-×)
+
+## Resources whose cost = amount × unit_price (GEMINI_USD is excluded ? it is already a USD
amount).
+_RATE_MULTIPLIED = (GPU_SECONDS, WALL_SECONDS, EGRESS_GB)
+
+
+def compute_cost_usd(usage: Mapping[str, float],
+                    rates: Mapping[str, float]) -> Tuple[float, Dict[str, float]]:
+    """Return ``(cost_usd, breakdown)`` for one job.
+
+    ``usage`` maps a resource key to its measured amount (seconds / GB); ``rates`` maps a
resource
+    key to its USD unit price (already resolved for this pricebook version + gpu_type by the DB
+    layer). ``GEMINI_USD`` in ``usage`` is added verbatim (it is already a USD cost, not
multiplied).
+    Missing amounts/rates default to ``0.0`` ? with the placeholder pricebook the total is
``0.0``.
+    """
+    breakdown: Dict[str, float] = {}
+    total = 0.0
+    for res in _RATE_MULTIPLIED:
+        amount = float(usage.get(res, 0.0) or 0.0)
+        rate = float(rates.get(res, 0.0) or 0.0)
+        cost = amount * rate
+        if amount or rate:                # record the line even at $0 so the breakdown is
transparent
+            breakdown[res] = round(cost, 6)
+        total += cost
+    gemini = float(usage.get(GEMINI_USD, 0.0) or 0.0)
+    if gemini:
+        breakdown[GEMINI_USD] = round(gemini, 6)
+    total += gemini
+    return round(total, 6), breakdown
+
+
+def apply_margin(cost_usd: float, margin: float) -> float:

```

```

+     """Resale price = ``cost_usd * (1 + margin)``. ``margin`` is clamped ? 0 (never below
cost)."""
+     return round(cost_usd * (1.0 + max(0.0, float(margin))), 6)
+
+
+def to_inr(cost_usd: float, fx_inr_per_usd: float) -> float:
+     """Display conversion: USD ? INR at the configured FX rate (2 dp, the display currency)."""
+     return round(float(cost_usd) * float(fx_inr_per_usd), 2)
diff --git a/backend/config/models.py b/backend/config/models.py
index 92ad880..960f6c7 100644
--- a/backend/config/models.py
+++ b/backend/config/models.py
@@ -351,6 +351,21 @@ class DeploymentConfig(BaseModel):
     compute_target: Literal["", "local_gpu", "cloud_run", "cpu_mock"] = ""

+class BillingConfig(BaseModel):
+     """Per-job cost ledger (COMPUTE_DECOUPLED_SERVING M8, D8).
+
+     **DEFAULT-OFF:** ``enabled=False`` ? nothing records cost (the v6 cost columns stay NULL =
+     today's behaviour). When enabled, a job's cost is computed + stored in **USD** via the
+     versioned
+     ``pricebook`` DB table and **displayed in INR** at ``fx_inr_per_usd``. The default
+     pricebook
+     version is the seeded ``placeholder-v0`` (every rate ``0.0`` ? cost ``0``) until the owner
+     inserts a real, calibrated version and points ``pricebook_version`` at it."""
+
+     enabled: bool = False
+     pricebook_version: str = "placeholder-v0" # the seeded $0 version; owner flips to a real
one
+     fx_inr_per_usd: float = 83.0 # USD?INR display rate (Bangalore market)
+     margin: float = 0.0 # resale markup (?0); 0 = bill at cost
+
+class AppConfig(BaseModel):
+     """Root configuration object.

@@ -372,6 +387,7 @@ class AppConfig(BaseModel):
     logging: LoggingConfig = Field(default_factory=LoggingConfig)
     storage: StorageConfig = Field(default_factory=StorageConfig)
     deployment: DeploymentConfig = Field(default_factory=DeploymentConfig)
+     billing: BillingConfig = Field(default_factory=BillingConfig)

     # Allow extra keys during transition so existing config.json files do not break.
     model_config = {
diff --git a/backend/main.py b/backend/main.py
index 6b89f6d..bc2ac10 100644
--- a/backend/main.py
+++ b/backend/main.py
@@ -39,6 +39,7 @@ from backend.config import ( # new typed config
from backend.results import Failure # standardized error/result contract
from backend.reporting import emit_indexer_report
from backend import storage # M2: URI-aware input acquisition (local = identity passthrough)
+from backend import billing # M8: per-job cost ledger (USD internal / INR display)

app = FastAPI(title="Sports Highlight Indexer API")

@@ -83,6 +84,7 @@ from backend.api.job_worker import ( # noqa: F401 (re-exports: names stay
addr
    _get_executor,
    _job_to_request,
    _MAX_DRAIN_WAKE_SEC,
+     _maybe_record_job_cost,
    _run_claimed_job,
    _schedule_next_drain_if_waiting,

```

```

)
@@ -306,6 +308,32 @@ def process_video(req: ProcessRequest):
    )

+@app.get("/api/jobs/{job_id}/cost")
+def get_job_cost(job_id: str):
+    """Per-job cost (COMPUTE_DECOUPLED_SERVING M8, D8). ``cost_usd`` is the source of truth
(matches
+    GCP billing); ``cost_inr`` is a display conversion at the configured FX rate. ``cost_usd``
is
+    null until billing records it (default-OFF / the placeholder pricebook ? 0)."""
+    cost = db_instance.get_job_cost(job_id)
+    if cost is None:
+        raise HTTPException(status_code=404, detail=f"Unknown job_id '{job_id}'")
+    fx = global_config.billing.fx_inr_per_usd
+    usd = cost.get("cost_usd")
+    cost["cost_inr"] = billing.to_inr(usd, fx) if usd is not None else None
+    cost["fx_inr_per_usd"] = fx
+    return cost
+
+
+@app.get("/api/videos/{video_id}/cost")
+def get_video_cost(video_id: str):
+    """Aggregate per-video cost across its jobs (M8): ``total_cost_usd`` + ``total_cost_inr`` +
the
+    per-job rows. A video is 1:1 with an infer job, but a re-ingest can produce several."""
+    agg = db_instance.get_video_cost(video_id)
+    fx = global_config.billing.fx_inr_per_usd
+    agg["total_cost_inr"] = billing.to_inr(agg["total_cost_usd"], fx)
+    agg["fx_inr_per_usd"] = fx
+    return agg
+
+
+@app.get("/api/jobs/{job_id}")
+def get_job(job_id: str):
+    """Job state: {status, progress, result, reports}. ``status`` = execution state

```

7. assistant (2026-06-21T11:10:46.003Z)

Now let me read the database migration and the test files.

8. user (2026-06-21T11:10:46.992Z)

```

diff --git a/backend/infrastructure/database.py b/backend/infrastructure/database.py
index ba7a59c..6ae4b87 100644
--- a/backend/infrastructure/database.py
+++ b/backend/infrastructure/database.py
@@ -4,6 +4,8 @@ import logging
     from contextlib import contextmanager
     from typing import List, Dict, Any, Optional, Tuple

+from backend import billing # pure cost math (no backend imports ? no cycle)
+
+logger = logging.getLogger(__name__)

 class Database:
@@ -102,7 +104,10 @@ class Database:

         if version < 5:
             self._migrate_to_v5(cursor)
-        # future: if version < 6: cursor.execute("ALTER TABLE ..."); PRAGMA user_version =
6
+
+        if version < 6:

```

```

+         self._migrate_to_v6(cursor)
+         # future: if version < 7: cursor.execute("ALTER TABLE ..."); PRAGMA user_version =
7
+
+         conn.commit()
+
@@ -284,6 +289,53 @@ class Database:
+         """
+         cursor.execute("PRAGMA user_version = 5")
+
+     def _migrate_to_v6(self, cursor):
+         # v6: per-job COST LEDGER (COMPUTE_DECOUPLED_SERVING M8, D8). NULLABLE cost columns on
+         # `jobs` (NULL = no cost recorded = byte-identical to today; nothing records until
+         # billing.enabled + a real pricebook) + a VERSIONED `pricebook` table. Cost is stored
in
+         # USD (matches GCP billing = one source of truth); INR is a display conversion at a
+         # configurable FX rate. ALTER (not recreate) so v1-v5 job rows survive; ADD COLUMNS are
+         # idempotent so an interrupted upgrade can re-run them. Additive ONLY ? no served
change.
+         for col, typ in (
+             ("owner_id", "TEXT"),           # who pays (the non-owner user / tenant)
+             ("compute_backend", "TEXT"),     # local_gpu | cloud_run | ...
+             ("gpu_type", "TEXT"),           # L4 | T4 | ... (selects the per-second GPU
rate)
+             ("gpu_active_seconds", "REAL"),
+             ("wall_seconds", "REAL"),
+             ("bytes_out", "INTEGER"),        # egress
+             ("gemini_cost_usd", "REAL"),     # provider-reported, already USD
+             ("pricebook_version", "TEXT"),
+             ("cost_usd", "REAL"),           # computed total (USD = source of truth)
+             ("cost_breakdown_json", "TEXT"), # {gpu_seconds: .., egress_gb: .., gemini_usd:
..}
+         ):
+             self._add_column_idempotent(cursor, f"ALTER TABLE jobs ADD COLUMN {col} {typ}")
+             # Partial index keeps the NULL=no-cost fast path churn-free while making per-owner
billing cheap.
+             cursor.execute("CREATE INDEX IF NOT EXISTS idx_jobs_owner ON jobs(owner_id) "
+                             "WHERE owner_id IS NOT NULL")
+             # Versioned pricebook: one row per (version, resource, gpu_type). Rates change + differ
by
+             # gpu_type ? re-version (never mutate a shipped version, so a recorded cost stays
auditable).
+             cursor.execute("""
+                 CREATE TABLE IF NOT EXISTS pricebook (
+                     version TEXT NOT NULL,
+                     resource TEXT NOT NULL,
+                     gpu_type TEXT NOT NULL DEFAULT '',
+                     unit_price_usd REAL NOT NULL DEFAULT 0.0,
+                     currency TEXT NOT NULL DEFAULT 'USD',
+                     effective_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
+                 )
+             """)
+             cursor.execute("CREATE UNIQUE INDEX IF NOT EXISTS idx_pricebook_key "
+                             "ON pricebook(version, resource, gpu_type)")
+             # Seed a PLACEHOLDER version with every rate 0.0 ? cost_usd computes to 0 (nothing
+             # user-facing changes). The owner INSERTs a real, calibrated version (real GCP $/GPU-s,
+             # egress $/GB) and flips billing.pricebook_version to it ? never overwrite this seed.
+             for resource, gpu_type in (("gpu_seconds", "L4"), ("gpu_seconds", "T4"),
+                                         ("wall_seconds", ""), ("egress_gb", "")):
+                 cursor.execute(
+                     "INSERT OR IGNORE INTO pricebook (version, resource, gpu_type, unit_price_usd,
+
+                     "currency) VALUES ('placeholder-v0', ?, ?, 0.0, 'USD')", (resource, gpu_type))
+             cursor.execute("PRAGMA user_version = 6")
+

```

```

def add_video(self, video_id: str, filepath: str, status: str, validation_results:
List[Dict[str, Any]]) -> bool:
    """Register or update a video row WITHOUT touching its child segments.

@@ -547,6 +599,84 @@ class Database:
    d["policy"] = json.loads(d["policy"]) if d.get("policy") else None
    return d

+ # --- Per-job cost ledger (COMPUTE_DECOUPLED_SERVING M8, D8) -----
+ def get_pricebook_rates(self, version: str, gpu_type: Optional[str] = None) -> Dict[str,
float]:
+     """Resolve a flat ``{resource: unit_price_usd}`` for a pricebook ``version``. The per-
second
+     GPU rate is selected by ``gpu_type``; non-GPU resources (wall/egress) carry
``gpu_type=''``.
+     An unknown version ? ``{}`` (so every cost computes to 0.0)."""
+     gt = gpu_type or ""
+     rates: Dict[str, float] = {}
+     with self._connect() as conn:
+         rows = conn.cursor().execute(
+             "SELECT resource, gpu_type, unit_price_usd FROM pricebook WHERE version = ?",
+             (version,)).fetchall()
+     for r in rows:
+         res, row_gt, price = r["resource"], (r["gpu_type"] or ""),
float(r["unit_price_usd"])
+         if res == billing.GPU_SECONDS:
+             if row_gt == gt:         # the GPU rate for THIS gpu_type
+                 rates[res] = price
+         else:
+             rates[res] = price      # wall/egress: gpu_type-independent
+     return rates

+ def record_job_cost(self, job_id: str, *, usage: Dict[str, float], pricebook_version: str,
+                     gpu_type: Optional[str] = None, compute_backend: Optional[str] = None,
+                     owner_id: Optional[str] = None, margin: float = 0.0) ->
Optional[Dict[str, Any]]:
+     """Compute the job's cost from ``usage`` x the pricebook + persist every cost column.
+     Returns the stored cost dict, or ``None`` on write failure. The caller gates this on
+     ``billing.enabled`` (default-OFF) ? nothing records until billing is turned on."""
+     rates = self.get_pricebook_rates(pricebook_version, gpu_type)
+     cost_usd, breakdown = billing.compute_cost_usd(usage, rates)
+     if margin:
+         cost_usd = billing.apply_margin(cost_usd, margin)
+     bytes_out = int(float(usage.get(billing.EGRESS_GB, 0.0) or 0.0) * 1e9)
+     ok = self._execute_write(
+         "UPDATE jobs SET owner_id=?, compute_backend=?, gpu_type=?, gpu_active_seconds=?, "
+         "wall_seconds=?, bytes_out=?, gemini_cost_usd=?, pricebook_version=?, cost_usd=?, "
+         "cost_breakdown_json=? WHERE id = ?",
+         (owner_id, compute_backend, gpu_type, usage.get(billing.GPU_SECONDS),
+          usage.get(billing.WALL_SECONDS), bytes_out, usage.get(billing.GEMINI_USD),
+          pricebook_version, cost_usd, json.dumps(breakdown), job_id),
+         f"Failed to record cost for job {job_id}")
+     if not ok:
+         return None
+     return {"cost_usd": cost_usd, "cost_breakdown": breakdown,
+           "pricebook_version": pricebook_version}

+ def get_job_cost(self, job_id: str) -> Optional[Dict[str, Any]]:
+     """The job's cost row (``None`` if the job doesn't exist; ``cost_usd`` is ``None``
until
+     recorded). ``cost_breakdown`` is the deserialized JSON breakdown."""
+     with self._connect() as conn:
+         row = conn.cursor().execute(
+             "SELECT id, video_id, owner_id, compute_backend, gpu_type, gpu_active_seconds,
"

```

```

+         "wall_seconds, bytes_out, gemini_cost_usd, pricebook_version, cost_usd, "
+         "cost_breakdown_json FROM jobs WHERE id = ?", (job_id,)).fetchone()
+     if not row:
+         return None
+     d = dict(row)
+     raw = d.pop("cost_breakdown_json", None)
+     d["cost_breakdown"] = json.loads(raw) if raw else {}
+     return d
+
+ def get_video_cost(self, video_id: str) -> Dict[str, Any]:
+     """Aggregate cost across a video's jobs: total `cost_usd` + the per-job rows. A video
is
+     1:1 with an infer job, but a re-ingest can produce several ? sum them."""
+     with self._connect() as conn:
+         rows = conn.cursor().execute(
+             "SELECT id, cost_usd, pricebook_version, cost_breakdown_json FROM jobs "
+             "WHERE video_id = ?", (video_id,)).fetchall()
+         jobs: List[Dict[str, Any]] = []
+         total = 0.0
+         for r in rows:
+             cost = r["cost_usd"]
+             if cost is not None:
+                 total += float(cost)
+             jobs.append({
+                 "job_id": r["id"], "cost_usd": cost, "pricebook_version":
r["pricebook_version"],
+                 "cost_breakdown": json.loads(r["cost_breakdown_json"]) if
r["cost_breakdown_json"] else {},
+             })
+         return {"video_id": video_id, "total_cost_usd": round(total, 6), "jobs": jobs}
+
+ # --- Self-scheduling execution policy (EXECUTION_POLICY.md P2) -----
def set_job_policy(self, job_id: str, policy: Optional[Dict[str, Any]]) -> bool:
    """Attach/replace a job's JSON ExecutionPolicy."""

```

9. assistant (2026-06-21T11:10:49.115Z)

Now let me read the test files in full.

10. user (2026-06-21T11:10:50.090Z)

```

1     """Per-job cost ledger (COMPUTE_DECOUPLED_SERVING M8, D8).
2
3     Pure cost math + the v6 migration (cost columns + versioned pricebook) +
record/get/aggregate +
4     the default-OFF wiring. Cardinal invariant: with the seeded ``placeholder-v0`` pricebook
(every
5     rate 0.0) every cost is 0, and with ``billing.enabled=False`` NOTHING records (cost
columns stay
6     NULL = today).
7     """
8     from backend import billing
9     from backend.config import AppConfig
10    from backend.infrastructure.database import Database
11
12
13    # ----- pure cost module
14
15    def test_compute_cost_zero_without_rates():
16        cost, bd = billing.compute_cost_usd({"gpu_seconds": 180, "egress_gb": 0.04}, {})
17        assert cost == 0.0 and bd == {"gpu_seconds": 0.0, "egress_gb": 0.0}
18
19
20    def test_compute_cost_with_rates_and_gemini_passthrough():
21        usage = {"gpu_seconds": 180, "egress_gb": 0.04, "gemini_usd": 0.05}

```



```

22     rates = {"gpu_seconds": 0.0002, "egress_gb": 0.10}
23     cost, bd = billing.compute_cost_usd(usage, rates)
24     assert cost == round(180 * 0.0002 + 0.04 * 0.10 + 0.05, 6) # 0.036 + 0.004 + 0.05
25     assert bd["gpu_seconds"] == 0.036 and bd["egress_gb"] == 0.004 and bd["gemini_usd"]
== 0.05
26
27
28     def test_apply_margin_clamps_and_to_inr():
29         assert billing.apply_margin(0.10, 0.20) == 0.12
30         assert billing.apply_margin(0.10, -5) == 0.10 # margin clamped to >= 0 (never
below cost)
31         assert billing.to_inr(1.0, 83.0) == 83.0
32
33
34     # ----- v6 migration +
pricebook seed
35
36     def test_v6_migration_adds_cost_columns_and_seeds_placeholder(tmp_path):
37         db = Database(str(tmp_path / "m.db"))
38         with db._connect() as conn:
39             ver = conn.execute("PRAGMA user_version").fetchone()[0]
40             cols = {r[1] for r in conn.execute("PRAGMA table_info(jobs)").fetchall()}
41             seeded = conn.execute(
42                 "SELECT unit_price_usd FROM pricebook WHERE
version='placeholder-v0'").fetchall()
43             assert ver >= 6
44             assert {"cost_usd", "gpu_active_seconds", "wall_seconds", "bytes_out", "owner_id",
45                 "pricebook_version", "cost_breakdown_json", "compute_backend", "gpu_type"}
<= cols
46             assert len(seeded) >= 4 and all(r[0] == 0.0 for r in seeded) # placeholder = every
rate $0
47
48
49     def test_v6_migration_is_idempotent(tmp_path):
50         """Re-opening the DB re-runs the ladder guard (version already 6 ? no-op, no
duplicate seed)."""
51         path = str(tmp_path / "m.db")
52         Database(path)
53         db2 = Database(path) # second open must not crash or duplicate the seed
54         with db2._connect() as conn:
55             n = conn.execute("SELECT COUNT(*) FROM pricebook WHERE
version='placeholder-v0'").fetchone()[0]
56             assert n == 4 # exactly the 4 seeded rows (INSERT OR IGNORE on the unique key)
57
58
59     # ----- pricebook rate
resolution
60
61     def _seed_real_pricebook(db, version="v1"):
62         with db._connect() as conn:
63             conn.executemany(
64                 "INSERT INTO pricebook (version, resource, gpu_type, unit_price_usd,
currency) "
65                 "VALUES (?, ?, ?, ?, 'USD')",
66                 [(version, "gpu_seconds", "L4", 0.0002, ), (version, "gpu_seconds", "T4",
0.0001, ),
67                 (version, "egress_gb", "", 0.10, ), (version, "wall_seconds", "", 0.0, )])
68             conn.commit()
69
70
71     def test_get_pricebook_rates_selects_by_gpu_type(tmp_path):
72         db = Database(str(tmp_path / "m.db"))
73         _seed_real_pricebook(db)
74         l4 = db.get_pricebook_rates("v1", "L4")
75         assert l4["gpu_seconds"] == 0.0002 and l4["egress_gb"] == 0.10

```

```

76         assert db.get_pricebook_rates("v1", "T4")["gpu_seconds"] == 0.0001
77         assert db.get_pricebook_rates("nonexistent", "L4") == {} # unknown version ? no
rates ? cost 0
78
79
80     # ----- record / get /
aggregate
81
82     def test_record_and_get_job_cost(tmp_path):
83         db = Database(str(tmp_path / "m.db"))
84         db.create_job("j1", "v.mp4")
85         db.set_job_video_id("j1", "vidA")
86         _seed_real_pricebook(db)
87         out = db.record_job_cost("j1", usage={"gpu_seconds": 180, "egress_gb": 0.04,
"wall_seconds": 200},
88                                     pricebook_version="v1", gpu_type="L4",
compute_backend="cloud_run",
89                                     owner_id="u1")
90         assert out["cost_usd"] == round(180 * 0.0002 + 0.04 * 0.10, 6)
91         cost = db.get_job_cost("j1")
92         assert cost["cost_usd"] == out["cost_usd"]
93         assert cost["gpu_type"] == "L4" and cost["owner_id"] == "u1" and
cost["compute_backend"] == "cloud_run"
94         assert cost["bytes_out"] == int(0.04 * 1e9)
95         assert cost["cost_breakdown"]["gpu_seconds"] == 0.036
96
97
98     def test_record_with_placeholder_pricebook_is_zero(tmp_path):
99         db = Database(str(tmp_path / "m.db"))
100         db.create_job("j1", "v.mp4")
101         out = db.record_job_cost("j1", usage={"gpu_seconds": 180},
pricebook_version="placeholder-v0",
102                                     gpu_type="L4")
103         assert out["cost_usd"] == 0.0 # default placeholder ? $0, nothing user-facing
changes
104
105
106     def test_record_applies_margin(tmp_path):
107         db = Database(str(tmp_path / "m.db"))
108         db.create_job("j1", "v.mp4")
109         _seed_real_pricebook(db)
110         out = db.record_job_cost("j1", usage={"gpu_seconds": 100}, pricebook_version="v1",
gpu_type="L4",
111                                     margin=0.20)
112         assert out["cost_usd"] == round(100 * 0.0002 * 1.20, 6)
113
114
115     def test_get_job_cost_unknown_returns_none(tmp_path):
116         assert Database(str(tmp_path / "m.db")).get_job_cost("nope") is None
117
118
119     def test_get_video_cost_aggregates_across_jobs(tmp_path):
120         db = Database(str(tmp_path / "m.db"))
121         for jid in ("j1", "j2"):
122             db.create_job(jid, "v.mp4")
123             db.set_job_video_id(jid, "vidA")
124             _seed_real_pricebook(db)
125             db.record_job_cost("j1", usage={"gpu_seconds": 100}, pricebook_version="v1",
gpu_type="L4")
126             db.record_job_cost("j2", usage={"gpu_seconds": 50}, pricebook_version="v1",
gpu_type="L4")
127         agg = db.get_video_cost("vidA")
128         assert agg["total_cost_usd"] == round(100 * 0.0002 + 50 * 0.0002, 6)
129         assert len(agg["jobs"]) == 2
130

```



```

+def test_job_cost_endpoint_after_recording_shows_inr(env):
+    client, db, _tmp, _mp = env
+    db.create_job("j1", "v.mp4")
+    with db._connect() as conn:
+        conn.execute("INSERT INTO pricebook (version, resource, gpu_type, unit_price_usd,
currency) "
+                        "VALUES ('v1', 'gpu_seconds', 'L4', 0.001, 'USD')")
+        conn.commit()
+    db.record_job_cost("j1", usage={"gpu_seconds": 100}, pricebook_version="v1", gpu_type="L4")
+    body = client.get("/api/jobs/j1/cost").json()
+    assert body["cost_usd"] == 0.1
+    assert body["cost_inr"] == round(0.1 * 83.0, 2) # USD?INR at the configured FX
+
+def test_video_cost_endpoint_aggregates(env):
+    client, db, _tmp, _mp = env
+    for jid in ("j1", "j2"):
+        db.create_job(jid, "v.mp4")
+        db.set_job_video_id(jid, "vidA")
+    with db._connect() as conn:
+        conn.execute("INSERT INTO pricebook (version, resource, gpu_type, unit_price_usd,
currency) "
+                        "VALUES ('v1', 'gpu_seconds', 'L4', 0.001, 'USD')")
+        conn.commit()
+    db.record_job_cost("j1", usage={"gpu_seconds": 100}, pricebook_version="v1", gpu_type="L4")
+    db.record_job_cost("j2", usage={"gpu_seconds": 50}, pricebook_version="v1", gpu_type="L4")
+    body = client.get("/api/videos/vidA/cost").json()
+    assert body["total_cost_usd"] == 0.15 and body["total_cost_inr"] == round(0.15 * 83.0, 2)
+    assert len(body["jobs"]) == 2

```

12. assistant (2026-06-21T11:10:53.640Z)

Now let me verify a few specifics. First, let me check the `\_add\_column\_idempotent` helper, the `get\_video\_cost` behavior when no rows exist (the 404/empty case), and whether `get\_job\_cost`'s SELECT column list matches the v6 columns exactly.

13. user (2026-06-21T11:10:55.632Z)

```

64: def _add_column_idempotent(cursor: sqlite3.Cursor, ddl: str) -> None:
65:     """Apply a bare ``ALTER TABLE ... ADD COLUMN`` migration step re-runably.
66:
67:     A DDL ``ALTER`` auto-commits independently of the surrounding ``with conn``
68:     transaction, so an interrupted v3/v4 block can leave the column added while
69:     ``user_version`` stays un-bumped ? and the next open would re-run the same
70:     ``ALTER`` and raise ``OperationalError: duplicate column name``, bricking every
71:     future ``Database(...)`` construction (``_init_db`` runs in ``__init__``). Swallow
72:     ONLY that specific error so the ladder stays crash-safe, matching the re-runnable
73:     ``CREATE ... IF NOT EXISTS`` v1/v2/v5 blocks.
74:     """
75:     try:
76:         cursor.execute(ddl)
77:     --
219:         self._add_column_idempotent(cursor, "ALTER TABLE jobs ADD COLUMN policy TEXT")
220:         self._add_column_idempotent(cursor, "ALTER TABLE jobs ADD COLUMN next_poll_at
TIMESTAMP")
221:         cursor.execute("CREATE INDEX IF NOT EXISTS idx_jobs_due ON jobs(status,
next_poll_at)")
222:         cursor.execute("PRAGMA user_version = 3")
223:
224:     def _migrate_to_v4(self, cursor):
225:         # v4: personalization groundwork (PERSONALIZATION_PLAN.md §2). Add a NULLABLE
226:         # `user_id` to videos + candidate_segments so a future per-user / multi-tenant
227:         # path can scope rows to an owner. **NULL = local install = byte-identical to
228:         # today** ? every existing row stays NULL and every existing query (which never
229:         # filters on user_id) is unaffected. ALTER (not recreate) so v1-v3 rows survive;

```

```

230-         # the ADD COLUMNs are applied idempotently so an interrupted upgrade can re-run
them.
231-         # Additive ONLY: no served-behavior change rides on this slice.
232-         self._add_column_idempotent(cursor, "ALTER TABLE videos ADD COLUMN user_id TEXT")
233-         self._add_column_idempotent(cursor, "ALTER TABLE candidate_segments ADD COLUMN
user_id TEXT")
234-         # Partial indexes (WHERE user_id IS NOT NULL) keep the NULL=local fast path free
235-         # of index churn while making the eventual per-user lookups cheap.
236-         cursor.execute(
237-             "CREATE INDEX IF NOT EXISTS idx_videos_user ON videos(user_id) "
238-             "WHERE user_id IS NOT NULL")
239-         cursor.execute(
240-             "CREATE INDEX IF NOT EXISTS idx_candidates_user "
241-             "ON candidate_segments(user_id) WHERE user_id IS NOT NULL")
242-         cursor.execute("PRAGMA user_version = 4")
243-
244-     def _migrate_to_v5(self, cursor):
245-         # v5: per-user model store (PERSONALIZATION_PLAN.md §2). One row per (user_id,
--
311-         self._add_column_idempotent(cursor, f"ALTER TABLE jobs ADD COLUMN {col} {typ}")
312-         # Partial index keeps the NULL=no-cost fast path churn-free while making per-owner
billing cheap.
313-         cursor.execute("CREATE INDEX IF NOT EXISTS idx_jobs_owner ON jobs(owner_id) "
314-             "WHERE owner_id IS NOT NULL")
315-         # Versioned pricebook: one row per (version, resource, gpu_type). Rates change +
differ by
316-         # gpu_type ? re-version (never mutate a shipped version, so a recorded cost stays
auditable).
317-         cursor.execute("""
318-             CREATE TABLE IF NOT EXISTS pricebook (
319-                 version TEXT NOT NULL,
320-                 resource TEXT NOT NULL,
321-                 gpu_type TEXT NOT NULL DEFAULT '',
322-                 unit_price_usd REAL NOT NULL DEFAULT 0.0,
323-                 currency TEXT NOT NULL DEFAULT 'USD',

```

14. user (2026-06-21T11:10:56.382Z)

```

28: def env(tmp_path, monkeypatch):
29-     db = Database(str(tmp_path / "t.db"))
30-     cfg = AppConfig()
31-     monkeypatch.setattr(m, "db_instance", db)
32-     monkeypatch.setattr(m, "global_config", cfg)
33-     monkeypatch.setattr(m, "_get_executor", lambda: _InlineExecutor())
34-     client = TestClient(m.app)
35-     return client, db, tmp_path, monkeypatch
36-
37-
38: def _video(tmp_path, name="m.mp4"):
39-     p = tmp_path / name
40-     p.write_bytes(b"not-a-real-video-but-hashable")
41-     return p
42-
43-
44: def _pass():
45-     return [ValidationResult(validator_name="x", passed=True, message="ok", details={})]
46-
47-
48: def _fail():
49-     return [ValidationResult(validator_name="blur", passed=False, message="blurry",
details={})]
50-
51-
52: class _FakeSeg:
53-     def __init__(self, db, cfg):

```

```

54-         self.db = db
55-
56-     def process_video(self, video_id, video_path, *a, **k):
57-         sid = self.db.add_play_segment(video_id, 0.0, 5.0)
58-         return [{"id": sid, "start_time": 0.0, "end_time": 5.0, "duration": 5.0, "metadata":
59-         {}}]

```

15. assistant (2026-06-21T11:10:59.719Z)

Now let me verify the column-list match for `get\_job\_cost`, and check the `get\_video\_cost` endpoint behavior for an unknown video (does it 404 or return zero?). Let me also confirm there's no test for the unknown-pricebook-version path through `record\_job\_cost` and check the `to\_inr(None)` guard. Let me run the relevant tests and lint/type checks.

16. user (2026-06-21T11:11:14.846Z)

```

..... [100%]
42 passed in 11.69s

```

17. user (2026-06-21T11:11:18.263Z)

All checks passed!

18. user (2026-06-21T11:11:19.525Z)

Success: no issues found in 3 source files

19. assistant (2026-06-21T11:11:25.254Z)

Lint and types are clean. Now let me verify the specific lens questions. First, the `get\_video\_cost` endpoint for an unknown video ? what does it return? The DB method returns `total\_cost\_usd: 0.0` with empty jobs (no 404). Let me confirm, and check whether `get\_job\_cost`'s SELECT column list exactly matches the v6 columns. The v6 columns added are: owner_id, compute_backend, gpu_type, gpu_active_seconds, wall_seconds, bytes_out, gemini_cost_usd, pricebook_version, cost_usd, cost_breakdown_json. The SELECT lists: id, video_id, owner_id, compute_backend, gpu_type, gpu_active_seconds, wall_seconds, bytes_out, gemini_cost_usd, pricebook_version, cost_usd, cost_breakdown_json. That matches exactly (plus pre-existing id, video_id). Good.

Let me check the unknown-video endpoint behavior and whether the pipeline emits `usage` today (lens question 4).

20. user (2026-06-21T11:11:27.815Z)

```

backend/api/job_worker.py:104:    (`result['usage']` = {gpu_seconds, wall_seconds, egress_gb,
backend/api/job_worker.py:116:        usage = (result or {}).get("usage") or {}
backend/api/job_worker.py:118:        job["id"], usage=usage,
pricebook_version=billing_cfg.pricebook_version,
Binary file backend/api/__pycache__/job_worker.cpython-313.pyc matches

```

21. assistant (2026-06-21T11:11:32.208Z)

Confirmed: nothing in the pipeline/main emits a `result['usage']` key today. So even when `billing.enabled=True`, `usage` is always `{}` and cost is always 0 until the worker is wired to emit usage. This is documented in the `\_maybe\_record\_job\_cost` docstring ("compute the job's cost from the usage the pipeline reported") but the docstring frames it as if the pipeline already reports usage. Let me check if this no-op-when-enabled fact is honestly documented anywhere. Let me look at the design doc and the result contract.

22. user (2026-06-21T11:11:34.756Z)

docs/TRACKNET_WSL_SETUP.md:1:# TrackNet / WASB on WSL2 ? setup & usage

docs/archives/checkpoints/2026-06-logging-reporting/README.md:20:- Any change to harvest, spec rules, or logger usage should be validated against this baseline.

docs/archives/checkpoints/2026-06-logging-reporting/README.md:25:- Code: backend/reporting/, backend/main.py (logging.basicConfig), pipeline/segmenters/* (logger usage).

docs/archives/CODE_AUDIT_AND_TEST_HARDENING-COMPLETED-2026-06-14.md:32:The audit began by reading the mandated entry points (`docs/README.md`, `docs/CODE\_MAP.md`, `CLAUDE.md`) followed by core modules (main.py, config, database, registries, segmenters, detectors, validators, stitcher, providers, reporting, results, etc.) and cross-cutting searches (hardcoded paths, config access patterns, Result usage, prints, registration side-effects).

docs/archives/CODE_AUDIT_AND_TEST_HARDENING-COMPLETED-2026-06-14.md:43:**Item 2: Config type vs. usage mismatch + loader behavior**

docs/archives/CODE_AUDIT_AND_TEST_HARDENING-COMPLETED-2026-06-14.md:143:- T9?T11: Ablation integration assessment + concrete hardening proposals + living doc work. **Risk: LOW** (mostly additive usage + docs).

docs/archives/CODE_AUDIT_AND_TEST_HARDENING-COMPLETED-2026-06-14.md:298:- Specific corrections noted and applied for #155 landing: Item 2/3/4 references tightened, command examples updated (e.g. protocol references aligned with --annotations usage), registry scopes clarified (sports + annotations paths), extra="allow" scoping and _unknown_key_paths behavior documented, "Review round" checklist block recorded.

docs/archives/CODE_AUDIT_AND_TEST_HARDENING-COMPLETED-2026-06-14.md:359:**This PR:** Positive tests for **Item 2 (Config type vs. usage mismatch + loader behavior)**.

docs/archives/MONETIZATION_AUDIT.md:121:- Gemini terms/data/pricing: `ai.google.dev/gemini-api/terms`, `.../docs/data-usage`, `.../docs/pricing`.

docs/archives/past_projects/DECOUPLED_COMPUTE/04-BUILD-AND-TEST-PLAN.md:22: - record usage (gpu_seconds, egress, gemini_calls) ? cost ledger (05)

---wiring note in code---

23. assistant (2026-06-21T11:11:44.534Z)

Let me check the design doc to see if the M8 tracker honestly notes that usage emission is a separate/future step, and check the worker docstring framing more carefully.

24. user (2026-06-21T11:11:46.245Z)

docs/archives/past_projects/DECOUPLED_COMPUTE/05-COST-ATTRIBUTION.md

docs/archives/past_projects/DECOUPLED_COMPUTE/DEPLOY_REPORT_GCP_2026-06-20.md

docs/archives/past_projects/DECOUPLED_COMPUTE/HANDOFF.md

docs/COMPUTE_DECOUPLED_SERVING/architecture.svg

docs/COMPUTE_DECOUPLED_SERVING/README.md

docs/COMPUTE_DECOUPLED_SERVING/runlogs/README.md

docs/COMPUTE_DECOUPLED_SERVING/TESTING_STRATEGY.md

docs/DOC_STATUS.md

docs/NEXT_STEPS.md

docs/SETUP_NEW_MACHINE.md

--- changelog/tracker ---

25. assistant (2026-06-21T11:11:49.396Z)

The diff includes no doc changes (no tracker/changelog update in the diff, unlike the M4/M3 PRs that updated `RALLY\_QUALITY\_RESEARCH.md` §7). Let me check the M8 tracker doc to see if the "usage not emitted yet" caveat is recorded there.

26. user (2026-06-21T11:11:50.699Z)

31:| D4 | In **cloud-serving, STITCH runs on Cloud Run** (co-located with detect) so CPU/wall + egress are metered into the per-job cost ledger. | owner (a) | Stitch-on-laptop would hide real cost. |

35:| D8 | **Pricebook = a versioned DB table**; cost computed in **USD** (matches GCP billing = one source of truth), **displayed in INR** at a configurable FX rate (Bangalore market); re-versioned when GCP prices change. | owner 2026-06-21 | M8. |

44:| D17 | **Real cloud verification BUDGET = \$100 / ?10k cap (owner 2026-06-21):** the owner authorized actual GCP spend for due-diligence + production-grade testing/verification of M5?M7 (real L4 run, the M5 runlog, the M7 DEPLOY_REPORT), **provided the cap is respected** ? confirm

exact cmd + hourly cost before each spend, prefer the smallest viable run + cheapest-adequate GPU, and **DELETE** every resource after** ([[gcp-vm-always-delete]]). Removes the M6/M7 "spend" gate within the envelope; the calibrated pricebook (M8) + counsel ToS (M9) + OAuth app (M12) stay owner/counsel-side. | owner 2026-06-21 | Unblocks the real M5/M7 runs (not just the code) within the cap. |

91:DETECT on GPU everywhere; WINDOW CPU-pure everywhere; STITCH is real CPU work. In **cloud-serving**, stitch is co-located in the Cloud Run job** so `gpu\_active\_seconds` (detect) + `wall\_seconds` (detect+stitch) + `bytes\_out` (reel egress) land on **one metered invocation** ? the per-job cost ledger. Running stitch on a laptop would leave that cost **invisible/unattributed** ? exactly what the owner wants to avoid. (Stitch stays synchronous in the GPU job for the common one-reel case; a queued CPU-only stitch fallback only if compiles get bursty ? the job table + `claim\_due\_job` already supports both.)

94:**Exists:** the 3-stage core; `\_resolve\_runner` (the single compute seam); `StubDetectorRunner` (`replay\_csv` = byte-exact \$0 anchor); `StorageBackend` ABC + 4 backends + `StorageRef.parse\_uri`; the worker CLI (fetch/put, `--config`/`--set`); the async job control plane (`jobs` table, `claim\_due\_job` CAS); `ExecutionPolicy WAIT\_AND\_RESUME`/`JobWaiting` (carries to Cloud Run dispatch verbatim); the chunked-upload router (`<2GB`); `skip\_ai\_handoff` (LocalNoAI); storage fakes; golden fixtures + experiment harness. **Partial:** per-video workspace helper (default-OFF); `DeviceContext` (designed); edge-auth (v4 `user\_id` columns exist). **Net-new:** the `deployment` section; `input\_backend`; configurable output base (kill hardcoded `output/`); Cloud Run dispatch shim + container image; cost ledger + pricebook; edge-auth header extraction; startup env detection.

123:| M8 | Per-job **cost ledger** (v_ migration: `owner\_id, gpu\_active\_seconds, wall\_seconds, bytes\_out, cost\_usd, cost\_breakdown\_json`) + **pricebook** + aggregation + `/api/.../cost` | M7 | ? owner (billing) | ? | ? |

125:| M10 | `byo\_worker` / zero-infra-BYO ? **design-only, DEFERRED** | M8,M9 | ? explicit owner approval | ? | ? |

150:- [] Cost ledger (M8) makes GPU + **stitch** + egress cost visible per job; edge auth (M9) gates multi-tenant.

164:- 2026-06-21 ? **M5+ batch authorized + design picks locked** (owner): the remaining \$8 picks resolved ? **D16** (M6 = Cloud Run **Jobs** + bucket-poll; **libx264** encode + NVDEC decode-only; **hard-reject** `<2GB` input cap; dir name confirmed) and **D17** (real cloud verification budget = **\$100/?10k cap**, delete-after). Owner gave a blanket go to write+merge the **default-OFF** code for every remaining gated milestone (M5,M6,M8,M9-auth,M9-consent-cols,M11-plumbing,M12), one isolated PR each, reserving the gate for the real run/spend/calibration/counsel. Grounded by gate-map workflow `wf\_4849400f`.

27. assistant (2026-06-21T11:11:58.965Z)

The M8 tracker line 123 still shows `?` (not started/done) and the diff includes no doc update flipping it to done or adding a changelog entry ? unlike prior milestones (M4 commit `44c9f01` explicitly did "mark \$7 tracker M4 ? + changelog"). That's a hygiene gap relative to the project's documented convention.

Now let me verify the critical lens-4 concern about whether the worker docstring honestly conveys that usage is not emitted today. The docstring says "compute the job's cost from the usage the pipeline reported (result['usage'] = {...})" ? phrased as if the pipeline reports it, but it doesn't. Let me also confirm there's no test for unknown-pricebook-version through `record\_job\_cost` (only through `get\_pricebook\_rates`), and verify the `to\_inr` guard handles the video endpoint. Let me check one more edge: `get\_video\_cost` for an unknown video returns total 0.0 (not 404) ? is that asserted/tested?

28. user (2026-06-21T11:12:00.812Z)

```
tests\test_api_endpoints.py:258:         "video_id": "vid", "segment_ids": [s_low, s_high,
s_unknown],
tests\test_api_endpoints.py:592:     body = client.get("/api/videos/vidA/cost").json()
tests\test_billing.py:119: def test_get_video_cost_aggregates_across_jobs(tmp_path):
tests\test_billing.py:127:     agg = db.get_video_cost("vidA")
tests\test_billing.py:128:     assert agg["total_cost_usd"] == round(100 * 0.0002 + 50 * 0.0002,
6)
tests\test_experiment.py:119: def test_load_golden_videos_rejects_unknown_audio_policy(tmp_path):
tests\test_upload_download.py:164: def test_download_unknown_video_404(env):
```


29. assistant (2026-06-21T11:12:11.791Z)

Confirmed: there is no test for `GET /api/videos/{unknown}/cost`. The endpoint returns `200` with `total\_cost\_usd: 0.0` for any unknown video id (no 404), which is an asymmetry with the job-cost endpoint (which 404s on unknown). This is a real, grounded API-contract observation worth noting ? and it's untested.

Let me confirm the `get\_video\_cost` endpoint indeed does not 404 and `to\_inr(0.0)` works, by checking the code path once more (already read: `get\_video\_cost` always returns a dict with `total\_cost\_usd`, never None, so the endpoint never 404s).

Now let me double check the lens-6 items: is there a test that `record\_job\_cost` with an unknown pricebook version yields 0? Only `get\_pricebook\_rates("nonexistent")` is tested at the rates level (line 77), not end-to-end through `record\_job\_cost`. And margin IS tested (line 106). The placeholder-0 path through record is tested (line 98). Real-rate path tested (line 82). Default-OFF tested (line 134) and enabled tested (line 147).

Let me verify one subtle correctness point: in `test\_maybe\_record\_job\_cost\_records\_when\_enabled`, the usage passes `gpu\_type` inside `usage` dict, and `\_maybe\_record\_job\_cost` reads `usage.get("gpu\_type")`. Let me confirm that GPU rate resolution would actually be non-zero if a real pricebook were used ? i.e., that the `gpu\_type` plumbing is exercised. The test uses placeholder (0), so a real-rate enabled-path through the worker hook is NOT tested. Let me verify the breakdown assertion gap and run the full migration idempotency concern around the partial assertion in test line 17.

30. user (2026-06-21T11:12:15.419Z)

zero-rate breakdown: {'gpu_seconds': 0.0, 'egress_gb': 0.0}
wall-only-zero breakdown: {}

31. assistant (2026-06-21T11:12:18.239Z)

I have enough grounded evidence. Let me do a final confirmation that the full test suite collects (no broken imports from the new `billing` module in `database.py`), and verify the `\_maybe\_record\_job\_cost` re-export doesn't break anything.

32. user (2026-06-21T11:12:30.356Z)

..... [100%]
56 passed in 9.53s

33. assistant (2026-06-21T11:12:35.002Z)

All tests pass, lint and types clean. I have completed my review. Let me compile the grounded findings.

Summary of verification: tests pass (42 in billing+api, 56 in db/jobs), `ruff` and `mypy` clean on all changed files. The ledger is tested both ways (placeholder-0 and real-rate; default-OFF and enabled), the API tests assert INR conversion + 404 + null-before-recording, and `get\_job\_cost`'s SELECT column list exactly matches the v6 columns. The real gaps are around honesty of the no-op-when-enabled fact, an untested API asymmetry, and a missing doc/tracker update.

34. user (2026-06-21T11:13:09.853Z)

Structured output provided successfully

35. assistant (2026-06-21T11:13:22.704Z)

I've completed the TEST RIGOR / API / HYGIENE review of the M8 cost-ledger change. Verification was fully reproducible locally: 42 tests pass in `test\_billing.py` + `test\_api\_endpoints.py`, 56 in the db/jobs suites, and both `ruff check` and `mypy` are clean on every changed file.

Key grounded findings reported via StructuredOutput:

1. ****(major)** Enabled billing is a silent no-op today** ? ``backend/api/job_worker.py:116`` reads ``result['usage']``, but nothing in the pipeline/main ever emits that key (verified by grep). So even with ``billing.enabled=True`` and a real pricebook, every cost stays 0 until usage emission is separately wired. The docstring frames it as already working, and the tests hand-inject ``result['usage']`` (which the real worker never produces), reinforcing the illusion. Not honestly documented.

2. ****(minor)** ``/api/videos/{id}/cost`` returns `200/cost=0` for an unknown video** (no 404), asymmetric with the job endpoint's 404, and the unknown-video case is untested.

3. ****(minor)** No end-to-end test** that an unknown pricebook version through ``record_job_cost`` yields 0 (only tested at the ``get_pricebook_rates`` layer).

4. ****(minor)** No real-rate (non-zero) cost test through the enabled worker hook** ? ``_maybe_record_job_cost``'s own argument plumbing (gpu_type/owner_id) is only exercised against the placeholder (\$0) pricebook.

5. ****(nit)** M8 doc tracker not flipped + no changelog** in the diff, deviating from the prior-milestone convention (the immediately preceding commit explicitly did this for M4).

On the lens checklist's positives: both placeholder-0 and real-rate paths ARE tested; default-OFF and enabled paths ARE tested; the API tests DO assert INR conversion, the 404, and null-before-recording; ``get_job_cost``'s explicit SELECT column list exactly matches the v6 columns (no typo); and margin IS tested. The v6 migration is additive-NULL + idempotent and that idempotency is tested.

The most relevant file is ``C:\Users\avidu\Projects\badminton-highlight-indexer\backend\api\job_worker.py`` (lines 101-122) for finding 1.